RSTAR HRMS — BACKEND

Access Control: How It Works

Tables, request flow, scope resolution, and the “add an employee” path · 26 Oct 2026

1 · The two identity tables

Everything below hangs off one of these two. Mixing them up is the single most expensive mistake in this schema.

dbo.tblEmployeeDetails — the root

`PK EmployeeDetailId BIGINT`

The person. Name, DepartmentId, ManagerId, DateOfJoining, IsActive. **Every grant, audit column and the JWT subject key on this id.**

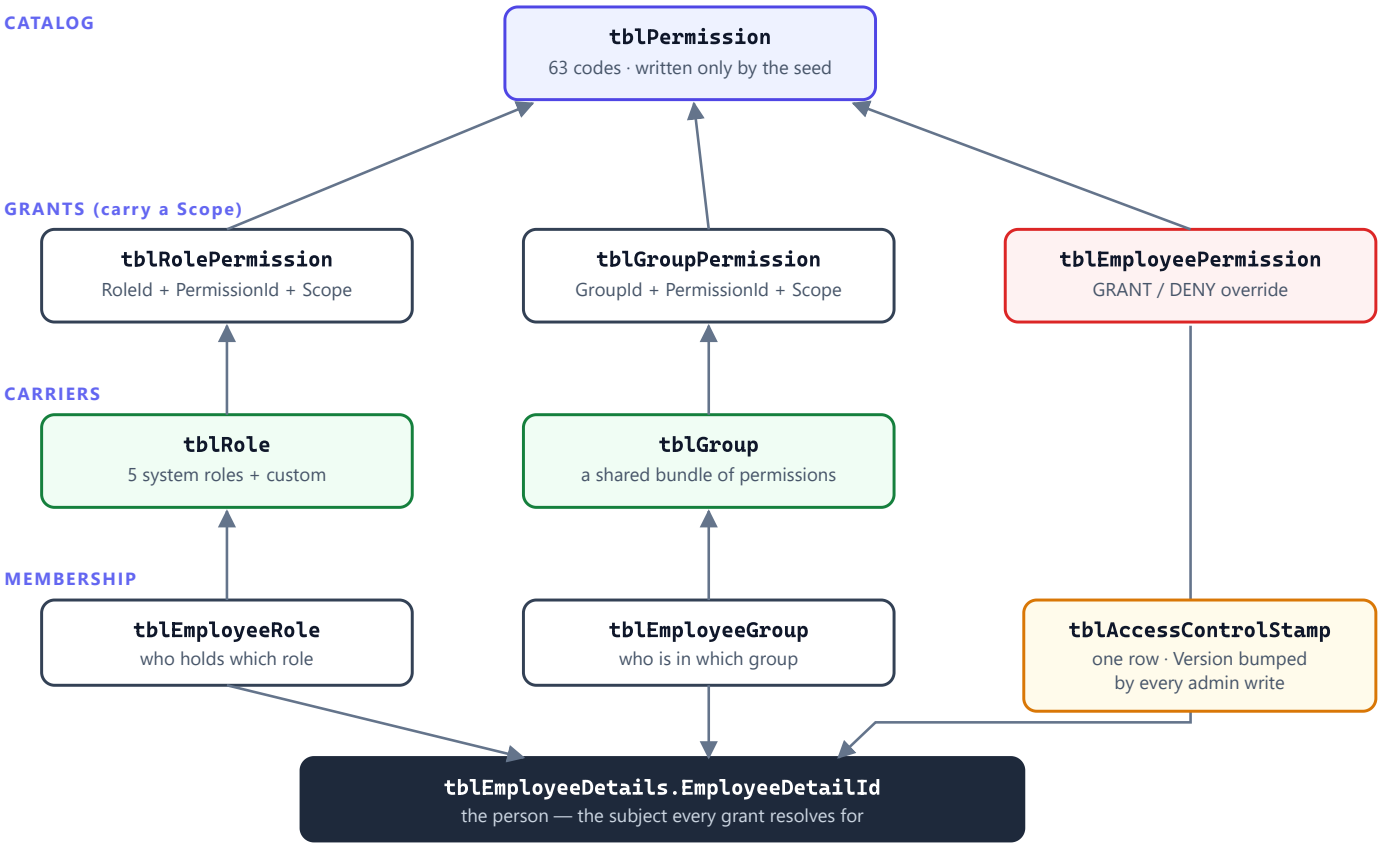
dbo.EmployeeLogin — a satellite

`PK EmployeeId INT`
`FK EmployeeDetailId NOT NULL`

Credentials — email, password hash, OTP. A credential id and nothing more. An employee may have **no** login; a login can never lack an employee.

One id space, by design. The first cut had this reversed: a nullable `EmployeeId` on the employee table, with grants keyed on the login. Every row-scoped query then had to translate between two id spaces, and getting it wrong compiled, ran, returned rows, and returned the **wrong people**. Inverting the relationship deleted that whole class of bug — `ManagerId`, `DepartmentId`, every grant and the token subject are all `EmployeeDetailId` now. The seed keeps the numbers out of step on purpose (login 1 is employee 3), so a stray `EmployeeId` fails loudly instead of coinciding.

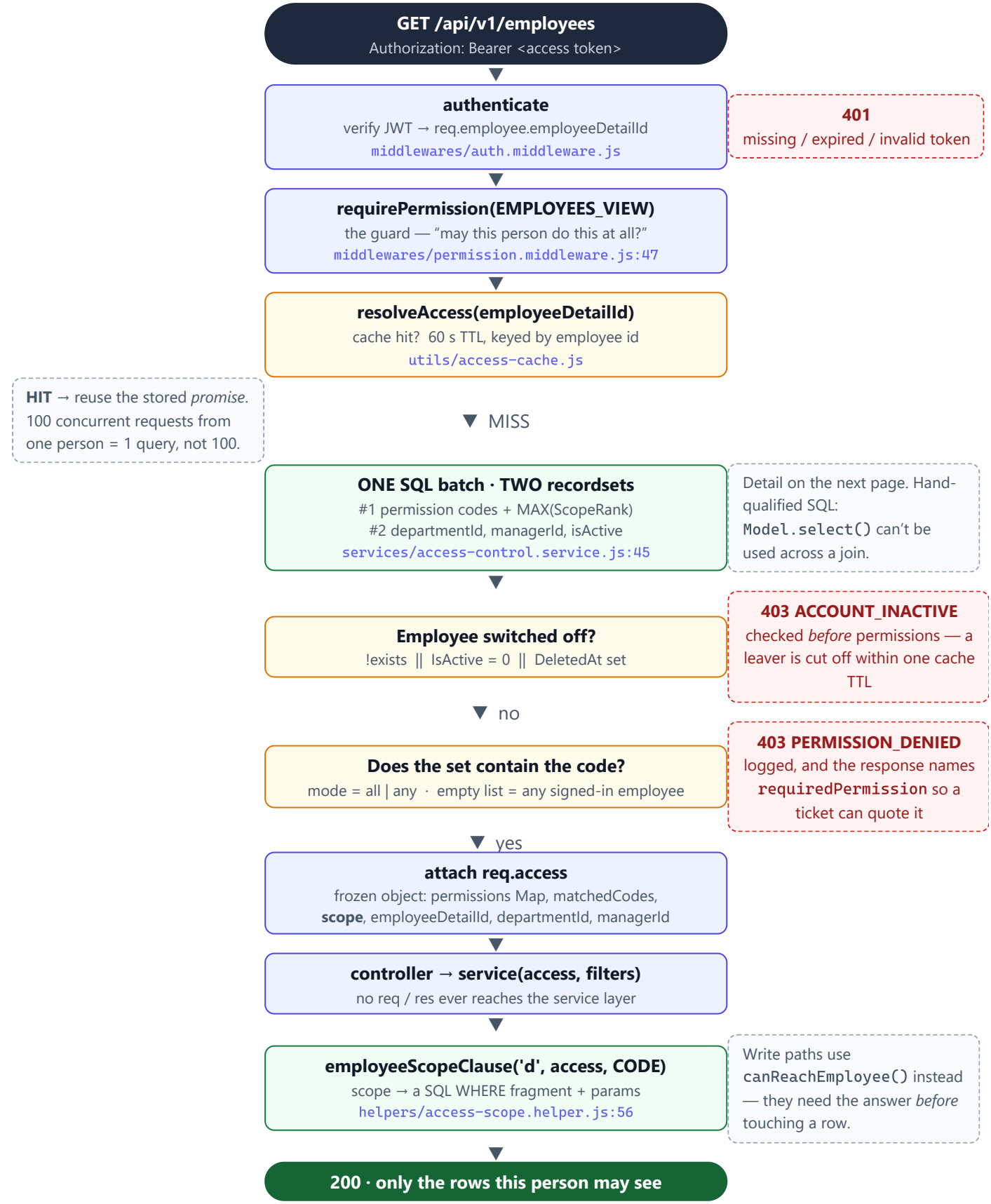
2 · The nine access-control tables



Every table is **soft-deleted** (`DeletedAt`), including the join tables — so “who gave Bob HR Admin, and when” stays answerable. Every read path must filter `WHERE DeletedAt IS NULL`. Groups carry **permissions only, never roles**: a second inheritance axis would bring the diamond problem for no capability a flat bundle doesn’t already give.

3 · One request, end to end

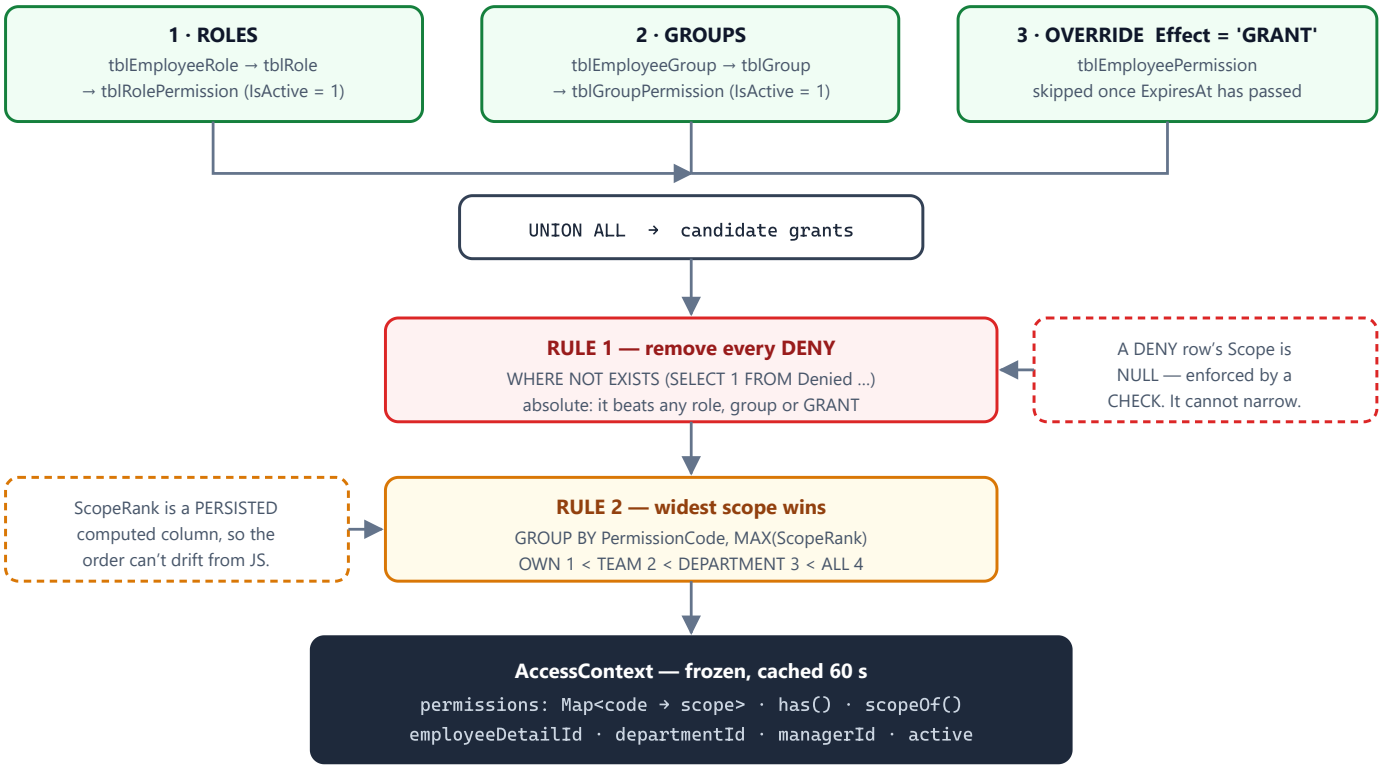
start / end | process | decision | rejected — request stops | side note



Division of labour. The middleware answers *"may you — at all?"*; the service plus the scope helper answers *"whose rows?"*. That is why the scope `switch` is never inlined into a service.

4 · Inside the resolver — how a permission set is computed

Three sources grant, one removes. It is a single round trip; the database does the precedence and the union.



The two rules are separate questions

Precedence decides **allow or deny**. Scope is a **union**, not a precedence chain — so an override `GRANT` at `OWN` does *not* shrink a role grant at `ALL`. To narrow somebody, take the wide role away.

Worked example (verified against the seeded roles)

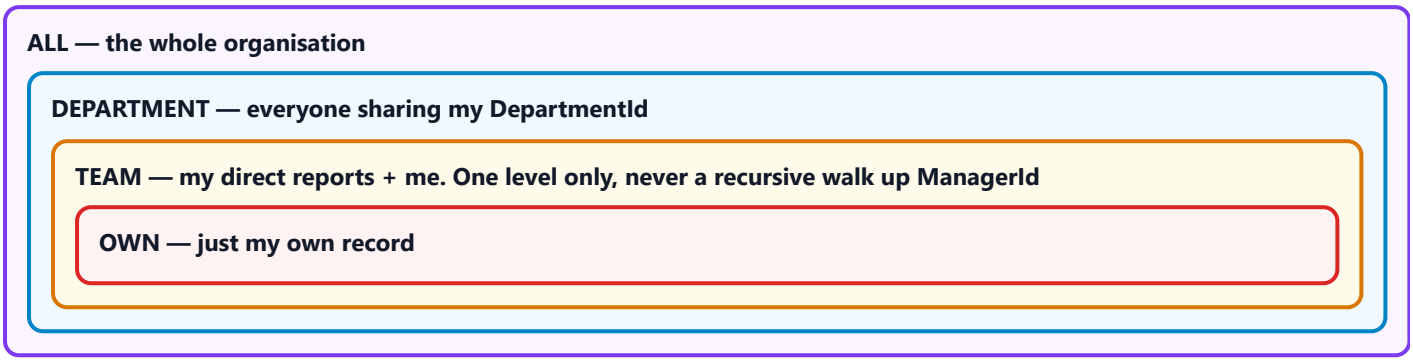
Subject's sources	total	employees.view	leave.approve	leave.apply	payroll.view
baseline — no roles	0	—	—	—	—
+ EMPLOYEE role	16	OWN	—	OWN	—
+ MANAGER role (union)	21	**TEAM**	TEAM	OWN	—
+ group grant payroll.view	22	TEAM	TEAM	OWN	**ALL**
+ DENY override on payroll.view	21	TEAM	TEAM	OWN	**—**
…once that DENY expires	22	TEAM	TEAM	OWN	ALL

Note `leave.apply` staying at `OWN` while `leave.view` rises to `TEAM` — different permissions resolve independently.

> **Super Admin holds real rows — there is no `if (isSuperAdmin) return next()` anywhere.** The seed grants it every live permission at `ALL`. The bypass was rejected because the account used for most manual testing would then never execute the resolver. The cost: a newly added permission 403s for Super Admin until `npm run db:seed:permissions` runs — loud, and one command to fix.

5 · Scope → the SQL that actually filters rows

The resolver says "you hold `employees.view` at TEAM". It does not know what TEAM means for an employee list versus a leave register — so every module gets **one clause builder** in `access-scope.helper.js`, and none of them is ever inlined into a service.



Scope	Emitted WHERE fragment	Keyed on
ALL	`1 = 1` — a literal, so the caller can always interpolate without wondering whether to add the AND	—
DEPARTMENT	`d.DepartmentId = @scopeDepartmentId`	access.departmentId
TEAM	`(d.ManagerId = @scopeDetailId OR d.EmployeeDetailId = @scopeDetailId)`	**EmployeeDetailId**
OWN	`d.EmployeeDetailId = @scopeDetailId`	**EmployeeDetailId**

> **Only DEPARTMENT can be unresolvable now.** `OWN` and `TEAM` key on `EmployeeDetailId`, which the subject always has — it *is* the subject. An employee with no `DepartmentId` still cannot answer DEPARTMENT, and the helper throws **403 SCOPE_UNRESOLVED** rather than returning a clause that matches nothing: an empty list reads as "there are no employees" and gets debugged as a data problem for an hour.

Read path vs write path

Reading — `employeeScopeClause()`

The clause goes straight into the `WHERE`. A row outside scope simply isn't in the result set, so no extra check is needed.

Writing — `canReachEmployee()`

Takes the loaded row and answers yes/no *before* the update or delete touches anything.

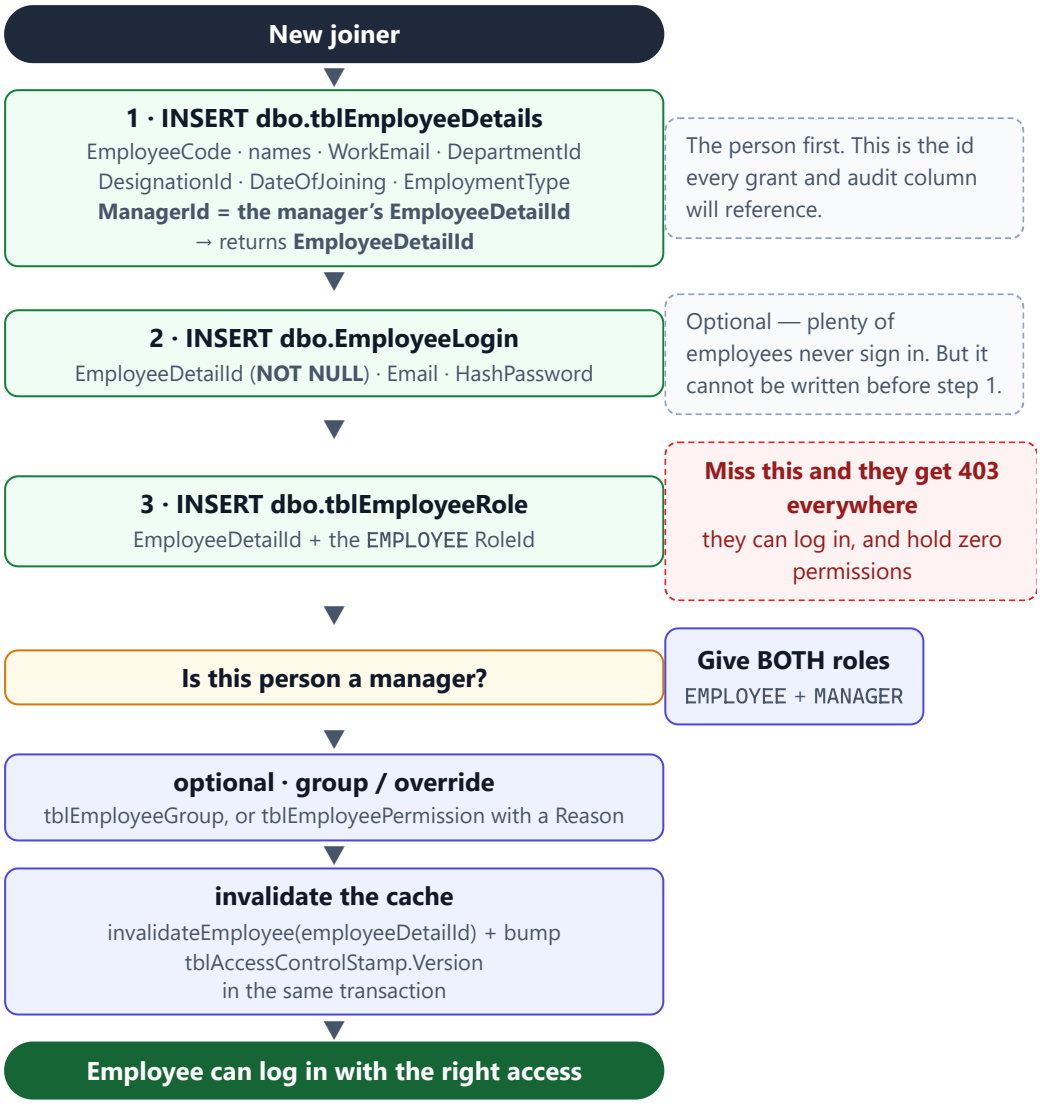
The seeded roles

Role	Grants	Notes
`SUPER_ADMIN`	56	Every permission at `ALL`. Re-synced on every seed run
`HR_ADMIN`	50	Full HR operations. Can assign roles but **not** edit what a role may do — assign-role plus assign-permission is a wildcard with extra steps
`HR_EXECUTIVE`	21	Day-to-day record maintenance
`EMPLOYEE`	16	Self-service only, at `OWN`
`MANAGER`	10	**Additive.** TEAM visibility and approvals, no self-service set

Per-user overrides are Super Admin only, and `Reason` is required by the API — an override bypasses the role model entirely, so it should be rare and attributable.

6 · Adding an employee — what actually has to happen

Status today: the Employees module is not built. `routes/index.js` still has `// router.use(ROUTE_GROUP.EMPLOYEES, employeeRoutes)` commented out, and the admin API for roles / groups / user-access does not exist yet. So right now this path runs through SQL or a seed script, not an endpoint.



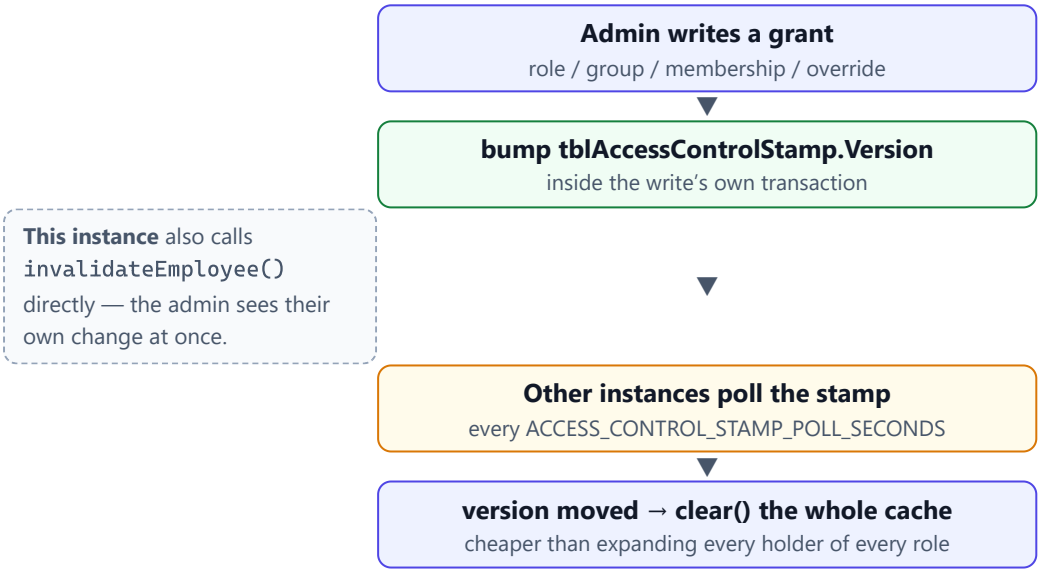
The most likely onboarding mistake. Assigning `MANAGER` *alone*. That role deliberately omits `leave.apply`, `payslip.view` and the rest of the self-service set — those come from `EMPLOYEE`, and the widest-scope union then produces `leave.view: TEAM` alongside `leave.apply: OWN`. A manager with only the Manager role cannot book their own leave.

`scripts/seed-employees.js` performs step 1 only, and `scripts/seed.js` steps 1–2. Neither assigns a role, so neither produces an account that can do anything. `npm run db:create:admin` is the one path that does all three in a single transaction.

6b · When POST /employees gets built — the checklist

1. **Uncommont the route group.** `ROUTE_GROUP.EMPLOYEES` and `ROUTES.EMPLOYEES` in `api-routes.enum.js`, then the `use()` in `routes/index.js`.
2. **Guard it.** `authenticate` first, then `requirePermission(PERMISSION.EMPLOYEES_CREATE)`. Use `requireAuthenticatedAccess()` rather than omitting a guard — "open to everyone" should be a decision in the route file, and the planned boot-time route audit treats a missing guard as a startup failure.
3. **Check reach on the write path.** `canReachEmployee(access, CODE, row)`, not the read-path clause.
4. **One transaction** for the login insert, the details insert and the default role row — plus the stamp bump.
5. **Use `OUTPUT INSERTED.*`.** `withTransaction`'s `run()` returns no `rowsAffected`, so without it an endpoint cheerfully reports success for a no-op.
6. **Never put `ScopeRank` in an INSERT column list** — SQL Server rejects writes to a computed column.

7 · The cache, and what it is not



Not built yet: `src/workers/access-stamp.worker.js`. Until it exists, other instances only pick a change up when their 60-second TTL expires. Single instance in development — fine. Worth closing before a multi-instance deploy.

Variable	Default	
`ACCESS_CONTROL_CACHE_ENABLED`	true	`false` makes every check hit the database — the fastest way to tell a permissions bug from a cache bug
`ACCESS_CONTROL_CACHE_TTL_SECONDS`	60	
`ACCESS_CONTROL_CACHE_MAX_ENTRIES`	1000	A hard bound, not an LRU — clears wholesale above this
`ACCESS_CONTROL_STAMP_POLL_SECONDS`	5	

The cache is not the revocation gap — the access token is. A fired employee's token stays valid for up to `JWT_ACCESS_EXPIRES_IN` (15 min) no matter what happens here. The mitigation that does exist: the guard rejects a profile that is `IsActive = 0` or soft-deleted, so deactivating somebody cuts them off within one cache TTL rather than one token lifetime.

8 · Changing things — the three common edits

You want to…	Do this
Add a permission	Add the code to `PERMISSION` → add its row to `PERMISSION_CATALOG` (pass `SCOPED` if it addresses somebody's rows) → decide defaults in `ROLE_PERMISSION_DEFAULTS` → `npm run db:seed:permissions`. Defaults only reach roles the seed *creates*; Super Admin always picks it up.
Change what a role may do	Through the admin API, **not** the enum. Storing grants as rows is exactly what makes this a non-deploy. The seed deliberately will not overwrite your change — `--reset-roles` forces it, and discards API edits.
Add a new scope	Three places must move together: `PERMISSION_SCOPE` + `PERMISSION_SCOPE_RANK`, the `CK_*_Scope` constraints and `ScopeRank` computed columns in a new migration, and the `switch` in `employeeScopeClause`. The duplication is deliberate — a new scope should be a decision, not a typo.

9 · Status

Built and verified

Migrations 001–005 applied from scratch · models, `db:verify` 14/14 · 63 permissions across 17 modules · resolver, cache, scope helper and guard, all keyed on `EmployeeDetailId` · `create-admin.js` · the Organization module · Postman and curl coverage

Not built yet

The admin API (roles, groups, catalog, `/access/my-permissions`) · the Employees module — the first consumer of row scope through an endpoint · the boot-time route audit · the stamp poller worker

Gotchas worth keeping in view

EmployeeId ≠ EmployeeDetailId	And you should never need both. `EmployeeLogin.EmployeeId` identifies a password row; everything else is `EmployeeDetailId`. The seed keeps them out of step on purpose (login 1 is employee 3), so a stray login id fails loudly rather than coinciding.
No IN / array support in the query helper	Generate `@p0..@pn` yourself, or pass a delimited string to `STRING_SPLIT` — safe here only because the code-shape CHECK forbids a comma. Bind it as `NVarChar(MAX)`; binding at the column width truncates the list and retires everything past the cut.
`Model.select()` can't cross a join	It emits unqualified column names and every table here has `CreatedAt` / `DeletedAt`. Multi-table reads are hand-qualified SQL.
Group membership has no department link	"Finance" is a manually maintained list. Someone who changes department stays in it. Not a bug today, but not a surprise later either.
sqlcmd needs `-I`	These tables carry filtered indexes and PERSISTED computed columns, so SQL Server refuses writes unless QUOTED_IDENTIFIER is ON. The driver and SSMS set it; sqlcmd does not.